In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [15]:
```
from sklearn import preprocessing
```

In [20]:
```
from sklearn.preprocessing import PolynomialFeatures
```

In [16]:
```
from sklearn.model_selection import train_test_split
```

In [17]:
```
from sklearn.linear_model import LinearRegression,Ridge,Lasso
```

In [21]:
```
from sklearn.metrics import r2_score
```

In [3]:
```
data = pd.read_csv(r"C:\Users\G BHARANIKA\Desktop\car-mpg.csv")
data.head()
```

Out[3]:

| | mpg | cyl | disp | hp | wt | acc | yr | origin | car_type | car_name |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 18.0 | 8 | 307.0 | 130 | 3504 | 12.0 | 70 | 1 | 0 | chevrolet chevelle malibu |
| **1** | 15.0 | 8 | 350.0 | 165 | 3693 | 11.5 | 70 | 1 | 0 | buick skylark 320 |
| **2** | 18.0 | 8 | 318.0 | 150 | 3436 | 11.0 | 70 | 1 | 0 | plymouth satellite |
| **3** | 16.0 | 8 | 304.0 | 150 | 3433 | 12.0 | 70 | 1 | 0 | amc rebel sst |
| **4** | 17.0 | 8 | 302.0 | 140 | 3449 | 10.5 | 70 | 1 | 0 | ford torino |

In [4]:
```
data = data.drop(['car_name'], axis = 1)
data['origin'] = data['origin'].replace({1: 'america',2: 'europe', 3: 'asia'})
data = pd.get_dummies(data,columns = ['origin'],dtype=int)
data = data.replace('?',np.nan)
```

In [47]:
```
data = data.apply(pd.to_numeric)

# Fill missing values with median only for numeric columns
numeric_cols = data.select_dtypes(include=[np.number]).columns
data[numeric_cols] = data[numeric_cols].apply(lambda x: x.fillna(x.median()))
```

# MODEL BUILDING

In [10]:
```
x= data.drop(['mpg'], axis = 1)
y = data[['mpg']]
```

In [11]:
```
x_s = preprocessing.scale(x)
x_s = pd.DataFrame(x_s,columns = x.columns)
```

```
y_s = preprocessing.scale(y)
y_s = pd.DataFrame(y_s, columns = y.columns)
```

In [18]:
```
#Split into train, test set
x_train, x_test,y_train,y_test = train_test_split(x_s, y_s,test_size = 0.30, random
x_train.shape
```

Out[18]: (278, 10)

2.SIMPLE LINEAR MODEL

In [32]:
```
regression_model = LinearRegression()
regression_model.fit(x_train, y_train)
print(x_train, y_train)
for idx, col_name in enumerate(x_train.columns):
    print('The coefficient for {} is {}'.format(col_name, regression_model.coef_[0]

intercept = regression_model.intercept_[0]
print('The intercept is {}'.format(intercept))
```

```
          cyl      disp        hp        wt       acc        yr  car_type  \
350 -0.856321 -0.849116 -1.081977 -0.893172 -0.242570  1.351199  0.941412
59  -0.856321 -0.925936 -1.317736 -0.847061  2.879909 -1.085858  0.941412
120 -0.856321 -0.695475  0.201600 -0.121101 -0.024722 -0.815074  0.941412
12   1.498191  1.983643  1.197027  0.934732 -2.203196 -1.627426 -1.062235
349 -0.856321 -0.983552 -0.951000 -1.165111  0.156817  1.351199  0.941412
..        ...       ...       ...       ...       ...       ...       ...
393 -0.856321 -0.513026 -0.479482 -0.213324  0.011586  1.621983  0.941412
255 -0.856321 -0.513026 -0.427091 -0.296088 -0.061030  0.538847  0.941412
72   1.498191  1.061796  1.197027  1.089620 -1.113959 -1.085858 -1.062235
235 -0.856321 -0.925936 -0.767632 -0.834055  0.955591  0.268063  0.941412
37   0.320935  0.370411 -0.112746  0.375483 -0.024722 -1.356642 -1.062235

     origin_america  origin_asia  origin_europe
350        0.773559    -0.497643      -0.461968
59        -1.292726    -0.497643       2.164651
120       -1.292726    -0.497643       2.164651
12         0.773559    -0.497643      -0.461968
349       -1.292726     2.009471      -0.461968
..              ...          ...            ...
393        0.773559    -0.497643      -0.461968
255        0.773559    -0.497643      -0.461968
72         0.773559    -0.497643      -0.461968
235       -1.292726     2.009471      -0.461968
37         0.773559    -0.497643      -0.461968

[278 rows x 10 columns]           mpg
350  1.432898
59  -0.065919
120 -0.578335
12  -1.090751
349  1.356035
..        ...
393  0.446497
255  0.203100
72  -1.090751
235  0.318393
37  -0.706439

[278 rows x 1 columns]
The coefficient for cyl is 0.321022385691611
The coefficient for disp is 0.32483430918483897
The coefficient for hp is -0.22916950059437569
The coefficient for wt is -0.7112101905072298
The coefficient for acc is 0.014713682764191237
The coefficient for yr is 0.3755811949510748
The coefficient for car_type is 0.3814769484233099
The coefficient for origin_america is -0.07472247547584178
The coefficient for origin_asia is 0.044515252035677896
The coefficient for origin_europe is 0.04834854953945386
The intercept is 0.019284116103639764
```

2.b Regularized ridge regression

In [31]:
```
ridge_model = Ridge(alpha = 0.3)
ridge_model.fit(x_train, y_train)
```

```
print(x_train)
print(y_train)

print('Ridge model coef: {}'.format(ridge_model.coef_))
```

```
          cyl      disp        hp        wt       acc        yr  car_type  \
350 -0.856321 -0.849116 -1.081977 -0.893172 -0.242570  1.351199  0.941412
59  -0.856321 -0.925936 -1.317736 -0.847061  2.879909 -1.085858  0.941412
120 -0.856321 -0.695475  0.201600 -0.121101 -0.024722 -0.815074  0.941412
12   1.498191  1.983643  1.197027  0.934732 -2.203196 -1.627426 -1.062235
349 -0.856321 -0.983552 -0.951000 -1.165111  0.156817  1.351199  0.941412
..        ...       ...       ...       ...       ...       ...       ...
393 -0.856321 -0.513026 -0.479482 -0.213324  0.011586  1.621983  0.941412
255 -0.856321 -0.513026 -0.427091 -0.296088 -0.061030  0.538847  0.941412
72   1.498191  1.061796  1.197027  1.089620 -1.113959 -1.085858 -1.062235
235 -0.856321 -0.925936 -0.767632 -0.834055  0.955591  0.268063  0.941412
37   0.320935  0.370411 -0.112746  0.375483 -0.024722 -1.356642 -1.062235

     origin_america  origin_asia  origin_europe
350        0.773559    -0.497643      -0.461968
59        -1.292726    -0.497643       2.164651
120       -1.292726    -0.497643       2.164651
12         0.773559    -0.497643      -0.461968
349       -1.292726     2.009471      -0.461968
..              ...          ...            ...
393        0.773559    -0.497643      -0.461968
255        0.773559    -0.497643      -0.461968
72         0.773559    -0.497643      -0.461968
235       -1.292726     2.009471      -0.461968
37         0.773559    -0.497643      -0.461968

[278 rows x 10 columns]
          mpg
350  1.432898
59  -0.065919
120 -0.578335
12  -1.090751
349  1.356035
..        ...
393  0.446497
255  0.203100
72  -1.090751
235  0.318393
37  -0.706439

[278 rows x 1 columns]
Ridge model coef: [[ 0.31649043  0.31320707 -0.22876025 -0.70109447  0.01295851  0.3
7447352
   0.37725608 -0.07423624  0.04441039  0.04784031]]
```

2.c REGULARIZED LASSO REGRESSION

In [36]:
```
lasso_model = Lasso(alpha = 0.1)
lasso_model.fit(x_train, y_train)

print('Lasso model coef: {}'.format(lasso_model.coef_))
```

```
Lasso model coef: [-0.         -0.         -0.01690287 -0.51890013  0.          0.28
138241
  0.1278489  -0.01642647  0.          0.        ]
```

3. SCORE COMPARISON

In [38]:
```
print(regression_model.score(x_train, y_train))
print(regression_model.score(x_test, y_test))

print('************************')
#Ridge
print(ridge_model.score(x_train, y_train))
print(ridge_model.score(x_test, y_test))

print('************************')
#Lasso
print(lasso_model.score(x_train, y_train))
print(lasso_model.score(x_test, y_test))
```

```
0.8343770256960538
0.8513421387780066
************************
0.8343617931312616
0.8518882171608507
************************
0.7938010766228453
0.8375229615977083
```

4.MODEL PARAMETER TUNING

In [41]:
```
data_train_test = pd.concat([x_train, y_train], axis =1)
data_train_test.head()
```

Out[41]:

| | cyl | disp | hp | wt | acc | yr | car_type | origin_ame |
|---|---|---|---|---|---|---|---|---|
| **350** | -0.856321 | -0.849116 | -1.081977 | -0.893172 | -0.242570 | 1.351199 | 0.941412 | 0.773 |
| **59** | -0.856321 | -0.925936 | -1.317736 | -0.847061 | 2.879909 | -1.085858 | 0.941412 | -1.292 |
| **120** | -0.856321 | -0.695475 | 0.201600 | -0.121101 | -0.024722 | -0.815074 | 0.941412 | -1.292 |
| **12** | 1.498191 | 1.983643 | 1.197027 | 0.934732 | -2.203196 | -1.627426 | -1.062235 | 0.773 |
| **349** | -0.856321 | -0.983552 | -0.951000 | -1.165111 | 0.156817 | 1.351199 | 0.941412 | -1.292 |

In [42]:
```
import statsmodels.formula.api as smf
ols1 = smf.ols(formula = 'mpg ~ cyl+disp+hp+wt+acc+yr+car_type+origin_america+origi
ols1.params
```

Out[42]:
```
Intercept         0.019284
cyl               0.321022
disp              0.324834
hp               -0.229170
wt               -0.711210
acc               0.014714
yr                0.375581
car_type          0.381477
origin_america   -0.074722
origin_europe     0.048349
origin_asia       0.044515
dtype: float64
```

In [43]:
```
print(ols1.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.834
Model:                            OLS   Adj. R-squared:                  0.829
Method:                 Least Squares   F-statistic:                     150.0
Date:                Thu, 21 Aug 2025   Prob (F-statistic):           3.12e-99
Time:                        14:50:35   Log-Likelihood:                -146.89
No. Observations:                 278   AIC:                             313.8
Df Residuals:                     268   BIC:                             350.1
Df Model:                           9
Covariance Type:            nonrobust
====================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------
Intercept          0.0193      0.025      0.765      0.445      -0.030       0.069
cyl                0.3210      0.112      2.856      0.005       0.100       0.542
disp               0.3248      0.128      2.544      0.012       0.073       0.576
hp                -0.2292      0.079     -2.915      0.004      -0.384      -0.074
wt                -0.7112      0.088     -8.118      0.000      -0.884      -0.539
acc                0.0147      0.039      0.373      0.709      -0.063       0.092
yr                 0.3756      0.029     13.088      0.000       0.319       0.432
car_type           0.3815      0.067      5.728      0.000       0.250       0.513
origin_america    -0.0747      0.020     -3.723      0.000      -0.114      -0.035
origin_europe      0.0483      0.021      2.270      0.024       0.006       0.090
origin_asia        0.0445      0.020      2.175      0.031       0.004       0.085
==============================================================================
Omnibus:                       22.678   Durbin-Watson:                   2.105
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               36.139
Skew:                           0.513   Prob(JB):                     1.42e-08
Kurtosis:                       4.438   Cond. No.                     1.59e+16
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly spe
cified.
[2] The smallest eigenvalue is 6.14e-30. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

In [44]:
```
mse  = np.mean((regression_model.predict(x_test)-y_test)**2)

# root of mean_sq_error is standard deviation i.e. avg variance between predicted a
```

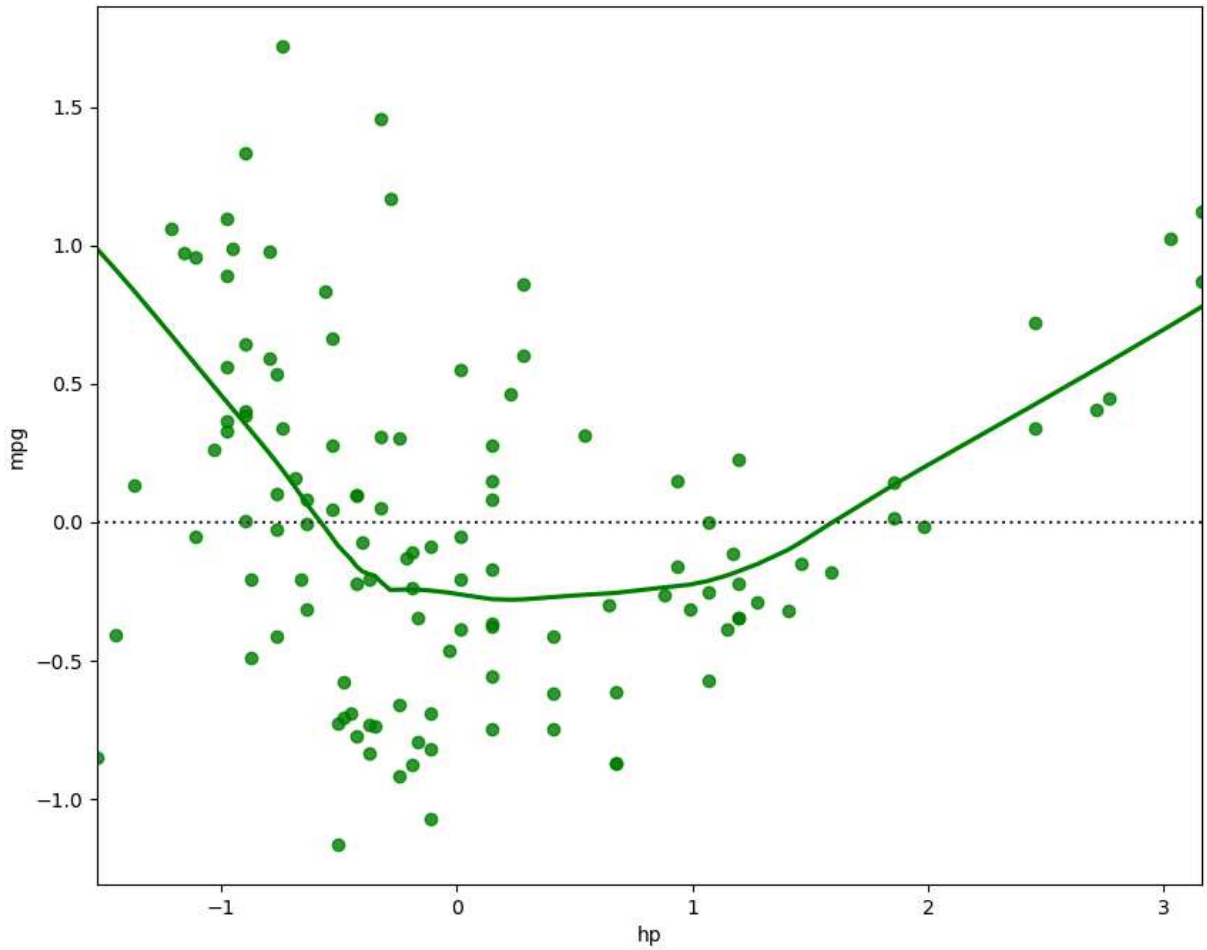
```
import math
rmse = math.sqrt(mse)
print('Root Mean Squared Error: {}'.format(rmse))
```

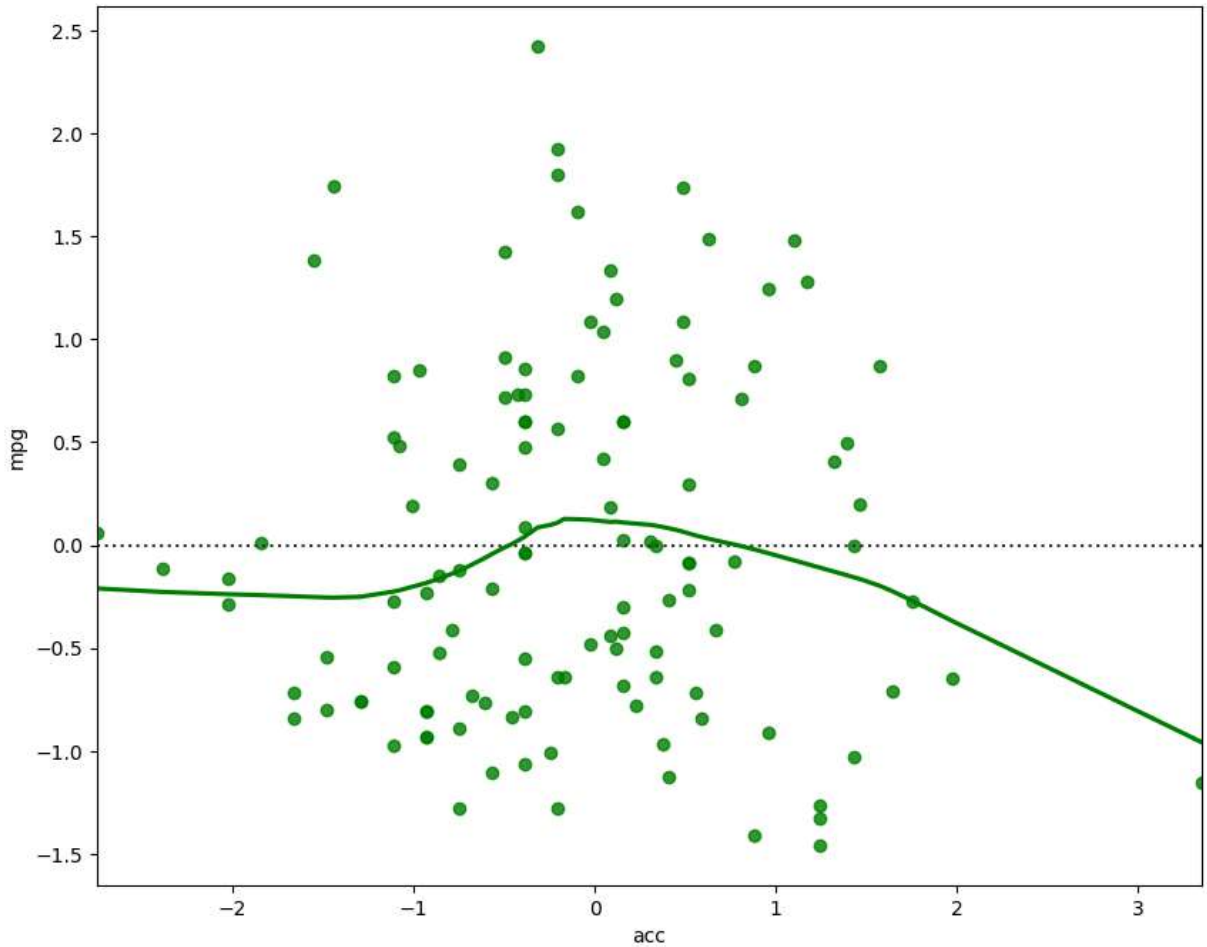
```
Root Mean Squared Error: 0.37766934254087847
```

In [45]:
```
fig = plt.figure(figsize=(10,8))
sns.residplot(x= x_test['hp'], y= y_test['mpg'], color='green', lowess=True )


fig = plt.figure(figsize=(10,8))
sns.residplot(x= x_test['acc'], y= y_test['mpg'], color='green', lowess=True )
```

Out[45]: <Axes: xlabel='acc', ylabel='mpg'>

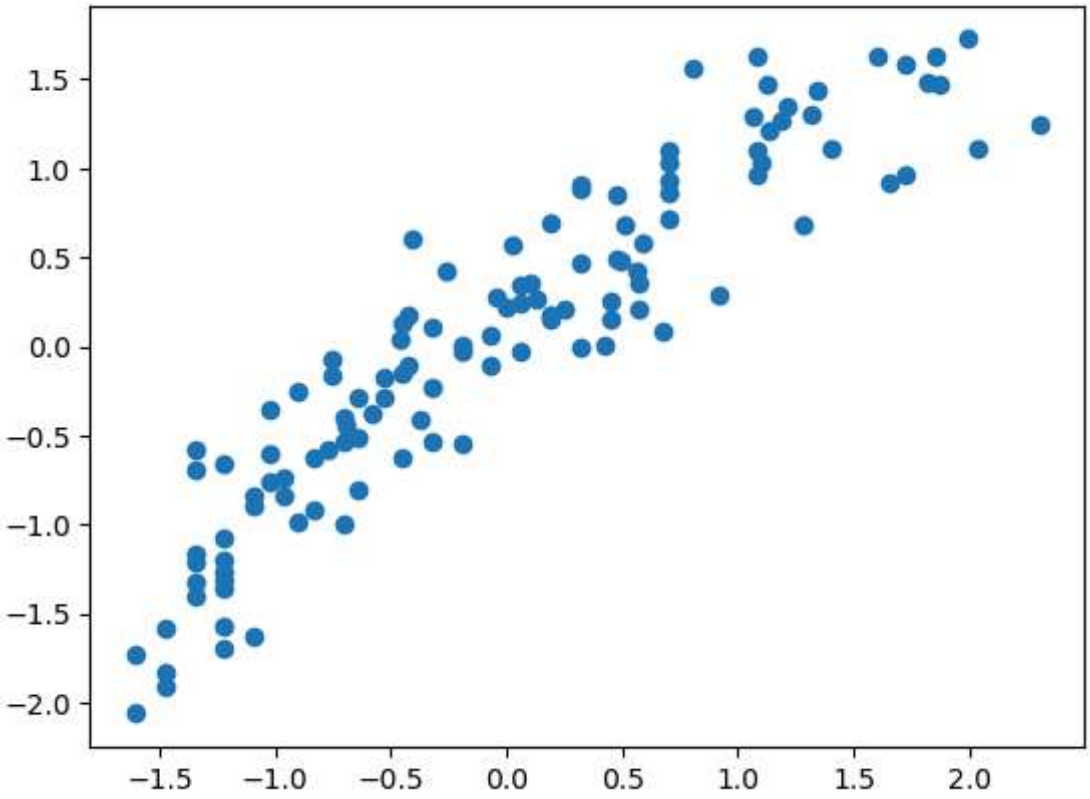





In [46]:
```
# predict mileage (mpg) for a set of attributes not in the training or test set
y_pred = regression_model.predict(x_test)

# Since this is regression, plot the predicted y value vs actual y values for the t
# A good model's prediction will be close to actual leading to high R and R2 values
#plt.rcParams['figure.dpi'] = 500
plt.scatter(y_test['mpg'], y_pred)
```

Out[46]:
```
<matplotlib.collections.PathCollection at 0x1e0954c5c70>
```




In [ ]: